Proxy Embeddings: What a Text Channel Can and Cannot Tell You About the Artifact

Abstract

The best text-to-image, text-to-video, text-to-music and text-to-code systems are closed. You cannot fine-tune them, you cannot reach their latents, and you cannot score a candidate in the space the artifact lives in without paying to generate it. The only control surface is a string, and the only cheap measurement is an embedding of that string. Working through a **proxy** is not a methodological choice in this setting; it is the setting.

This paper is about what that proxy can and cannot do. We measure it on five decoders that are deterministic and total — Python programs run on a fixed input battery, SQL against a fixed database, regular expressions against a fixed corpus of strings, arithmetic expressions on a grid, and SVG parsed to the marks it draws — so artifact distance is a property of the text alone and repeats exactly. The answers differ sharply by what you are asking the proxy for.

Hitting one specified target: the proxy works, and it is the strongest result here. Stating a required artifact-space property in the text and verifying by decoding, compliance is **62.5%** [54.2, 70.1] on generated programs and **97.8%** [94.4, 100.0] on regular expressions, against **6.9%** and **4.4%** for a decoy arm that states a *different* requirement in identical words. The decoy scores below unconditioned prompting, so the channel transmits which target rather than merely transmitting specificity. Reach extends past the generator's own experience: on targets it has **never once produced**, compliance is 94.4% for expressions and 25.0% for programs, against 0.0% for both controls.

And one line of prompt changes what you get back, though not in the way we first reported. Stating an emergent target as a band rather than a point lifts compliance per call issued from 27.3% to **78.8%** in the rarest band. But the arms do not answer at equal rates, and conditional on the generator answering at all the two are indistinguishable (97.7% against 97.3%): **a tolerance buys a returned artifact, not a better one.** Observability is the axis that moves accuracy — an unobservable target was answered 24 times out of 24 and hit 0 — and the two are separated in §3.8. The tolerance must still be in the *ask*: relaxing the acceptance criterion afterwards recovers nothing, because failures on emergent targets are wholesale rather than near-misses.

Matching a distribution: it works, and it degrades predictably. Compliance falls monotonically as the target moves into the generator's tail — 100%, 63.9%, 55.6%, 30.6%, 25.0% across five rarity bands on programs — which turns ”can I steer this?” into a budget question. The decay is governed by the *kind* of target rather than by rarity as such: targets the generator can write into the artifact stay high at every rarity, including outside anything it has ever produced, and only targets that must emerge from its behaviour decay. Inside a single decoder, holding everything else fixed, the two kinds differ by **47.7 points** [+34.8, +60.2].

Anything that reads pairwise distance: it fails, and the standard check does not detect the failure. Deduplication, coreset selection, farthest-point diversity, retrieval and contamination screening all consume a text-space distance as a stand-in for an artifact-space one. Qualifying that substitution by correlating the two is confounded: running the identical analysis between two *encoders*, with no decoder anywhere, reproduces the effect more strongly (near-minus-far +0.742 to +0.781 against the decoder's +0.386). Across three decoders and five encoders, **no decoder profile clears its encoder-to-encoder baseline distribution**. Downstream, deduplication ranks duplicate pairs at AUC 0.779 while **45.4% of the pairs it would reject are behaviourally far apart**, and greedy max-min loses to random on one decoder and beats it on another.

So spend your decode budget instead of your embedding budget. For cheap decoders the artifact-space oracle costs less than the embeddings it replaces (29.1 ms against 107.9 ms per item) and buys 1.54× [1.29, 1.85] the coverage at small budgets. For expensive decoders — video, audio, anything metered — the same logic holds partially: decoding a fraction of the pool and selecting on it recovers most of the oracle's advantage, and the fraction is small.

We ship `decodergap`, which reports the decoder-free control beside every correlation, scores each decision in the artifact's own space, and plans how to spend a fixed decode budget. The final section reports what all of this looks like on decoders that cannot be run cheaply at all, where text-embedding similarity explains 14.5% of the variance in rendered-image similarity and a third of the conditioning is lost at the boundary where one model's language becomes another model's input.

---

1. Introduction

Ask for a video and you will not get to see the latent space it was made in. The strongest text-to-image, text-to-video and text-to-music systems are served behind an API, are not open-weight, and accept exactly one kind of instruction: a string. Nothing about the artifact is available to the pipeline that produced the string except by generating the artifact and looking at it, which costs money and time proportional to how good the model is.

So the pipeline works in the modality it can touch. It writes text, and it measures text. It embeds its candidate prompts and makes decisions with those distances — drop the near-duplicates, keep the k most mutually distant, retrieve the nearest neighbours of a query, check a new batch against an old one for overlap — because an embedding is cheap and the artifact is not. Every one of those decisions is a bet that the geometry of the control modality stands in for the geometry of the output modality.

This paper measures the bet. Not in the abstract: on **decoders that are deterministic and total**, where the artifact of a string can be computed exactly, repeatably, and without a model in the loop. A Python function has a behaviour on a fixed battery of inputs. A SQL query returns rows. A regular expression accepts a set of strings. An arithmetic expression is a curve. An SVG document draws marks. Each of these is a real generation target with a real industry behind it, and each supplies something the interesting expensive cases cannot: a ground truth about the artifact that no embedding was consulted to produce.

The results do not compose into a verdict on proxies. They compose into three different verdicts, one for each thing people ask a proxy to do.

Individual targets. Can you make the artifact have a specific property by saying so in the text? Yes, and by a large margin over a control that states a different property in the same words. This is the paper's strongest positive result and it is the one that matters most in the closed-weight setting, because it is the only thing a text-only interface can be asked to do that does not require measuring anything.

Distributions. Can you steer the artifact distribution — cover a space, match a target, reach the tail? Partly, and how well depends on where you are aiming relative to what the generator already does. Compliance falls monotonically from certain to roughly one in four as the target moves out of the generator's habitual range, which converts a yes/no question into a budget.

Anything that reads a distance. Deduplication, diversity selection, coreset construction, retrieval, contamination screening. Here the proxy fails, and worse, the standard method of checking whether it fails does not work: the correlation between text distance and artifact distance has a confound that reproduces the entire effect with no decoder involved at all.

The distinction that organizes all three is between using the channel as a **lever** and as a **ruler** — to push the generator somewhere, or to measure how far apart two things are. The same embedding, the same corpus, the same generator: the first use survives every control we can construct and the second fails all of them.

The hinge between them is a number we nearly ignored. Permuting the artifacts across items destroys the text-to-artifact association completely, in every domain. **The information is there.** What fails is reading it off a metric defined on the text. A channel can be informative without its geometry being a measurement, and that is the sentence this paper exists to establish.

Why deterministic decoders. Diversity and coverage work on generated corpora routinely reports artifact-space numbers computed over rendered images. Those numbers are bounded by something no amount of care removes: hosted image endpoints expose no seed, so a render cannot be repeated and render noise cannot be separated from the quantity being measured. Every artifact-space claim in that setting is a statement about an unlogged seed distribution. Deterministic, total decoders remove the bound. A Python program on a fixed battery, a query against a fixed database, a pattern against a fixed corpus of strings, an expression on a grid, an SVG parsed to the marks it draws: each repeats exactly, so artifact distance is a property of the text alone. The expensive, non-repeatable case is not abandoned — it is §7 — but it is not where the load-bearing measurements are made.

What we found by being wrong. The first version of this work reported a mechanism: embeddings as near-field instruments, tracking the artifact among near-duplicates and saying nothing beyond. It did not survive its own control. Four further claims died the same way, every one built on the shape of a correlation, while every claim built on running a decision and scoring it in the artifact's space has held. §10 reports that pattern in full, because it is this paper's thesis applied to this paper's own methodology, and it is the best evidence we have that the confound is the default rather than an exotic case.

Contributions:

- **The lever result** (§3), on two decoders: a stated target is hit 62.5% and 97.8% of the time against 6.9% and 4.4% for matched decoys, reaching targets the generator has never produced at 94.4% and 25.0% against 0.0% for both controls.
- **A compliance curve against target rarity** (§3.2), which is what turns distributional steering into a budget.
- **The confound** (§4): correlational proxy validation reproduced in full by two encoders with no decoder, across three decoders and five encoders, with the control that detects it.
- **Decision verdicts** (§4.5–§4.7): aggregate deduplication supported, per-item refused at a 45.4% false-positive rate, diversity selection sign-unstable between decoders.
- **The decode budget** (§5): what a partial decode buys, with bootstrapped intervals, and the cost comparison that makes the oracle affordable for cheap decoders.
- **`decodergap`** (§6), which runs all of it and plans a fixed decode budget.
- **The expensive-decoder case** (§7): rendered images and audio, where none of the cheap advice applies.

2. Setup

Generator and embedders. `openai/gpt-5.6-luna` via OpenRouter for generation, judging, attractor mining and axis elicitation. Text embeddings: `nomic-embed-text` (768-d, unit-normalized) served locally by Ollama, so embedding is free and the loop is never rate-limited by its own measur-

ing instrument. Images: `gpt-image-1-mini`, embedded with CLIP ViT-B/32. Audio: Lyria 3 Pro instrumentals, embedded with CLAP and MERT. Every corpus is append-only JSONL with embeddings checkpointed alongside, resumable after a kill; every number carries a provenance tag naming the procedure that produced it.

Domains. *DALL·E instructions for post-modern artworks*, where diversity is the product and the same corpus can be measured in three spaces — words, text-embedding, and the rendered picture. *Psychometric test questions*, multiple-choice items assessing general reasoning, where two items with the same construct are a validity problem rather than an aesthetic one. *Instruction corpora*, for the head-to-head against released artifacts. *Poetry*, a pilot where craft rather than content carries the variation. *Instrumental-music prompts*, the longest chain in the paper: latent axes → text prompt → two-minute instrumental → audio embedding.

Methods compared. Five published approaches, implemented faithfully rather than as strawmen, each getting the same generator, embedder and budget accounting as ours.

arm	what it is
`naive`	plain repeated prompting — the honest floor
`high_temp`	temperature 1.6 — the trivial diversity lever
`self_instruct`	**Self-Instruct / Alpaca**: few-shot exemplars sampled from the pool, plus ROUGE-L ≤ 0.7 rejection against it
`evol_instruct`	**Evol-Instruct (WizardLM)**: sample an existing item, apply a random evolution operator (deepen, concretize, add constraint, harder reasoning, mutate form)
`persona`	**Persona-Hub / AttrPrompt**: a flat catalogue of personas × attributes, sampled uniformly
`rac`	**Recursive Axis Conditioning** (ours)
`rac+vision`	RAC, plus steering on the *rendered image* (§7.6.1)

What we measure, and what each measure misses. We report several because they disagree, and a corpus that one of them calls healthy another calls collapsed.

- **Exact-duplicate rate** — byte-identical repetition. Unambiguous, cheap, and the first thing to compute; blind to paraphrase.
- **Distinct-*n*, 4-gram self-repetition, *n*-gram Vendi** — surface statistics over token sequences. They catch templating that the duplicate rate misses and read meaning not at all. **These are independent of our objective**, since the method sees only embeddings and has no access to token counts or string identity.
- **Mean-centered embedding Vendi** — the exponential of the von Neumann entropy of the corpus Gram matrix, an effective number of distinct items. It summarizes the whole spectrum, which makes it insensitive to local structure. Centering matters as much as the statistic: the shared mean direction of text embeddings compresses the uncentered score by roughly 2.7×, so an uncentered Vendi reports the cone as much as the content.
- **Median nearest-neighbour distance** — how much room the typical item has. It goes to exactly zero when the median item has a perfect twin.
- **Worst-case nearest-neighbour distance (the packing radius)** — the only measure here under which a single collision is a defect regardless of everything else.
- **Coverage, density, precision and recall against a reference** — computed with reference-side k-NN radii so nothing is tunable per corpus. These are the only measures that know what the space is supposed to look like, and the only ones that let corpora of different scales be compared. Covered fraction at a *fixed* radius does not survive that comparison: it correlates -0.991 with within-corpus spacing, so it ranks corpora by how tightly they cluster rather than by how much they reach.
- **A judged threshold**, where an application defines the failure.
- **Measures on the rendered artifact**, and **measures in a channel the loop never optimizes** (§7.1), which are the ones that cannot be circular. **Scale and cost.** The text corpora comprise 43,171 real generations for roughly $8.50 of OpenRouter spend, plus 1,796 rendered images

across sixteen image arms and up to three quality tiers, a further 285 renders for the two render-side probes of §7.6.3 and §7.7, 100 rendered Lyria instrumentals, and 236 adjudicated exam-item pairs. Both `naive` arms reach $n = 10{,}000$; the reimplemented baselines reach 2,500 each. Every comparison is reported at a matched n that all compared arms actually reached. Total API spend across the project is roughly $40.

3. Individual targets: the channel as a lever

The rest of this paper uses the control modality as a *ruler* — a space in which to measure how far apart two items are — and every use of it in that role fails to clear a control. This section uses the same channel for the other thing it can do: carry an instruction. The permutation nulls of §4.3 establish that the information is present; the question here is whether it can be put in rather than read out.

This is the question that matters most when the generator is closed. A text-only API cannot be fine-tuned, cannot be conditioned on an embedding, and cannot be asked to optimize anything. It can be told what the artifact should do. Whether that works, and how far it reaches, decides what is achievable at all.

3.1 Stating the target, and verifying by decoding

A **target** is one checkable property of the artifact: a required output for a specified input (*`f([1, 2, 3])` must return 7*), a value that must appear in a query's result, a string a pattern must match. It is a fact about the artifact, it can be written into the text, and whether the artifact satisfies it is settled by decoding rather than by asking the model.

Four arms at one generator call each, on two decoders:

arm	what the prompt says	programs	SQL	regular expressions
`blind`	the fixed prompt; the target is never mentioned	18.1% [12.5, 24.3]	10.0% [6.1, 14.4]	2.2% [0.0, 5.6]
`decoy`	a *different* target, stated in identical form	**6.9%** [2.8, 11.1]	**1.7%** [0.0, 3.9]	**4.4%** [1.1, 8.9]
`instruct`	the target	**62.5%** [54.2, 70.1]	**52.8%** [45.0, 60.0]	**97.8%** [94.4, 100.0]
`retry`	`instruct`, then re-prompt showing the wrong value	61.8% [54.2, 69.4]	—	—

Counting a generation that returned nothing as a failure, which is what a practitioner issuing one call per target experiences. The arms do not complete at equal rates — on programs the blind prompt returned a usable program 99.3% of the time against `instruct`'s 63.2% — so the same numbers conditional on the model having answered are:

arm	programs	SQL
`blind`	18.2% [11.9, 24.5]	11.2% [6.9, 16.2]
`decoy`	12.5% [6.2, 20.0]	2.5% [0.0, 5.9]
`instruct`	**98.9%** [96.7, 100.0]	**79.2%** [71.7, 86.7]

The attrition runs *against* the result — instructed generations are the ones that most often come back empty — so the intent-to-treat figures understate the lever and both framings support it. Conditional on answering at all, a stated target is hit essentially always on programs. Only the intent-to-treat figures are quoted elsewhere in this paper, because they are the conservative ones.

Compliance with 95% bootstrap intervals; 60 targets × 3 seeds on programs and on SQL, 30 × 3 on expressions. Every interval for `instruct` is disjoint from its decoy's.

The decoy arm is what makes this a result rather than an observation. Adding any concrete requirement to a prompt changes what a generator produces, so `instruct` beating `blind` would be consistent

with the channel carrying nothing but specificity. The decoy is drawn from the same target pool and stated in the same words, so the two prompts differ in almost nothing but which value is demanded.

On programs the decoy scores below unconditioned prompting — 6.9% against 18.1%. Stating the wrong target does not merely fail to help; it moves the generator away from the right answer, below the rate reached with no instruction at all. The channel transmits *which* target, not that a target exists.

Re-prompting buys nothing: 61.8% on 149 calls against 62.5% on 144. Showing a model its own wrong answer and asking again does not recover the misses, which suggests the failures are targets the generator cannot construct rather than ones it carelessly missed.

3.2 Reaching what the generator has never produced

The interesting end of this is the tail. Each target's **rarity** is the share of the generator's own natural corpus — produced under the unconditioned prompt — that already satisfies it, so rarity is measured in the artifact distribution being steered rather than assumed. The rarest band is constructed rather than sampled: properties the generator produced **not once** in the natural corpus, but which are reachable in principle.

rarity of the target	programs	SQL	expressions
never produced	0.0 / 0.0 / **25.0**	0.0 / 0.0 / **72.2**	0.0 / 2.8 / **94.4**
[0.001, 0.02)	2.8 / 2.8 / **30.6**	0.0 / 0.0 / **25.0**	0.0 / 8.3 / **100.0**
[0.02, 0.10)	5.6 / 2.8 / **55.6**	0.0 / 0.0 / **44.4**	8.3 / 0.0 / **100.0**
[0.10, 0.30)	5.6 / 5.6 / **63.9**	8.3 / 0.0 / **36.1**	0.0 / 0.0 / **100.0**
[0.30, 1.01)	58.3 / 16.7 / **100.0**	41.7 / 8.3 / **86.1**	33.3 / 0.0 / **100.0**

*Percentages, as blind / decoy / **instruct**.*

The lever reaches outside the generator's demonstrated range. On properties never once produced under the unconditioned prompt, both controls score zero on programs and on SQL, while instruction reaches 25.0%, 72.2% and 94.4% on the three decoders. A pipeline restricted to sampling and filtering cannot obtain these artifacts at any budget, because the rate it is filtering is zero. Asking obtains them.

3.3 What governs how far it reaches: constructible against emergent

The three decoders disagree by a factor of four in the tail, and SQL's curve is not even monotone in rarity — 72.2% on never-produced targets against 25.0% one band up. Rarity is therefore not the governing variable. What the target *is* governs it.

A regular expression required to match `2024-01-31` can be **written** to match it: the requirement decomposes into syntax the generator is already producing. A SQL query required to surface `city=Lyon` can have a `WHERE` clause added. Neither requires the generator to discover anything, which is why compliance is high and nearly flat in rarity, and why SQL's never-produced band — column values that exist in the database but that no generated query happened to return — is its *easiest* band rather than its hardest.

A program required to return exactly 169 on a given thirteen-element list cannot be written that way. The value has to *come out* of whatever the function computes, and the generator has to find a computation that is both a plausible general-purpose function and lands on that number. That is a search, and it fails most of the time in the tail. SQL's mid-rarity bands behave the same way, because those targets are aggregate values — sums, counts, averages — that emerge from a computation rather than being nameable in a clause.

So the distinction that predicts reach is not rare against common but **constructible against emergent**:

- A **constructible** target is satisfied by writing something specific into the artifact. Compliance is high and roughly flat in rarity, including outside anything the generator has produced. Aim freely.

- An **emergent** target falls out of the artifact's behaviour rather than being written into it. Compliance decays with rarity — 100% to 25% on programs — and the budget per target must rise accordingly.

The distinction can be tested inside one decoder, and it survives. Compared across decoders it is confounded with everything else that differs between a regex and a program. SQL supplies the within-decoder test, because its targets span both kinds while generator, corpus, encoder, decoder and metric are all held fixed. The classification is mechanical and fixed by the database schema rather than by any outcome: a target column that *is* a column of the base schema can be named in a `WHERE` clause and is constructible; anything else is an alias for a computed expression whose value must come out of an aggregation, and is emergent.

SQL targets	`blind`	`decoy`	`instruct`	*n*
constructible (base-schema column)	16.0%	2.5%	**79.0%** [70.4, 87.7]	81
emergent (computed alias)	5.1%	1.0%	**31.3%** [22.2, 40.4]	99

The gap is **+47.7 points, 95% CI [+34.8, +60.2]**. Rarity survives as a smaller effect inside each class — the correlation between rarity and compliance is +0.251 ($p = 0.024$) among constructible targets and +0.252 ($p = 0.012$) among emergent ones — so both variables matter and the kind of target dominates.

This classification was applied after the run, though the rule that defines it reads only the schema. The prediction it makes for a decoder we had not yet measured was registered before that run rather than after; see `experiments/decodergap/PREREGISTRATION.md`.

Rarity in the natural corpus predicts compliance *within* an emergent domain and predicts it poorly across domains or across target kinds, which is worth knowing before planning a budget from one curve. The generalization to the closed-weight setting is direct: *a red door on the left of frame* is constructible; *the pacing of this edit rhythm* is emergent, and the second needs the budget the decay curve implies. Before spending on a target, ask which kind it is — the answer is usually obvious, and it changes the budget by a factor of several.

Three caveats belong with these numbers. Compliance is checked on the commanded property only, so an artifact that satisfies the target by special-casing it counts as a hit even though the prompt forbids it. The natural corpus and the targets come from the same generator, so a different generator would relabel which targets are rare. And `retry` was measured on programs only.

3.4 Ask for a range: it buys a returned artifact, not a better one

Emergent targets are the expensive ones, and there is a one-line change that recovers most of what they cost. State the requirement as a band rather than a point.

Same 48 integer targets, same decoder, same generator, one difference in the ask: *must return exactly 169* against *must return a value between 152 and 186* (±10% of the value, minimum ±2). Because a band is a looser success criterion as well as a looser ask, all four combinations of how the target is *asked* and how it is *scored* are reported.

asked	scored	programs	SQL (emergent targets)
exactly	exactly	66.7% [58.9, 74.4]	35.6% [25.6, 45.6]
as a band	**on the band**	**85.3%** [79.1, 91.5]	**60.0%** [50.0, 70.0]
as a band	exactly	32.6% [24.8, 41.1]	12.2% [5.6, 18.9]
exactly	on the band	67.4% [59.7, 75.2]	37.8% [27.8, 47.8]

Two decoders and two artifact spaces. The SQL arm is restricted to emergent targets by the schema rule of §3.3, since a band around a city name is meaningless. Every cell counts a generation that returned nothing as a failure — see below, because that choice turns out to carry the entire main effect.

The main effect is a return-rate effect, not an accuracy effect. The two ask types do not complete at the same rate: on programs the exact ask returned a decodable program 68.2% of the time against the band ask's 87.6%, a spread of 19.4 points, and on SQL 44.4% against 65.6%. Attrition that correlates with the treatment can manufacture an effect on its own, so every cell is recomputed conditional on the model having answered at all:

asked	scored	programs, of those that answered	SQL, of those that answered
exactly	exactly	**97.7%** [94.3, 100.0]	80.0% [67.5, 92.5]
as a band	on the band	**97.3%** [93.8, 100.0]	91.5% [84.7, 98.3]
as a band	exactly	37.2% [28.3, 46.9]	18.6% [10.2, 28.8]
exactly	on the band	98.9% [96.6, 100.0]	85.0% [72.5, 95.0]

On programs the tolerance benefit **disappears**: 97.7% against 97.3%. Asking for a band does not make the generator more accurate. It makes it more likely to answer at all, and the 18.6-point intent-to-treat gain is entirely the difference in how often an exact ask produces nothing usable. On SQL a residual gap survives (80.0% against 91.5%) but the intervals overlap and we do not claim it.

Both framings are legitimate and they answer different questions. A practitioner who issues one call per target and takes what comes back is in the intent-to-treat column, and for them the band ask really is worth 18.6 points — because a third of exact asks return nothing and that is a real cost. A practitioner who retries until the model answers is in the conditional column, and for them the band buys nothing on programs. **What we withdraw is the claim that a tolerance makes the artifact more likely to hit the target; what stands is that it makes the generator more likely to produce an artifact.**

We did not notice this until a collaborating session found the same failure in its own experiment and suggested checking completion rates by arm. The check is now part of the tool (§6) and of §10.

And by where the target sits in the generator's distribution:

rarity	asked exactly	asked as a band	lift
[0.001, 0.02)	27.3%	**78.8%**	**+51.5**
[0.02, 0.10)	55.6%	72.2%	+16.7
[0.10, 0.30)	95.8%	91.7%	−4.2
[0.30, 1.01)	94.4%	100.0%	+5.6

Intent-to-treat, so these inherit the return-rate effect above and should be read as "what one call per target buys", not as accuracy.

Per call issued, tolerance buys nothing in the head and 51.5 points in the tail. In the common bands compliance is already saturated; in the rare band a band takes a target from failing three times in four to succeeding four times in five. Given the correction above, the mechanism is that rare exact targets are the ones an exact ask most often fails to answer at all, and the band ask gets an answer out. The prescription survives with its reason changed: loosen the targets you are struggling to hit, because otherwise you will frequently get nothing back.

The two cross terms carry the caveats, and they are the reason all four cells were measured.

A band is not free. Asking for a band and then insisting on the exact value gives 32.6% on programs and 12.2% on SQL, in both cases *below* asking exactly (66.7% and 35.6%). The generator spends the slack it is given. If the point value genuinely matters, asking for a range around it makes things worse.

Failures are not near-misses. Asking exactly and scoring on the band gives 67.4% against 66.7% scored exactly on programs, and 37.8% against 35.6% on SQL — a difference of 0.7 and 2.2 points. When an exact ask fails on an emergent target it fails wholesale rather than landing nearby, on both decoders. So the tolerance has to be in the *ask*: relaxing the acceptance criterion after generation

recovers almost nothing, because there is almost nothing sitting just outside the line. §3.7 shows this is a property of *emergent* targets specifically, and that the opposite failure profile is what an unobservable criterion looks like.

That last point is the practical one. A pipeline that generates against exact targets and then accepts near-misses is getting the worst of both — it pays the exact ask's failure rate and gains none of the band's compliance. Put the tolerance in the prompt.

3.5 Three axes, and what to do about each

The reach of the lever is governed by three properties of the target, and they are separable. Each was isolated by a different measurement, and one of them was isolated by a failure.

Is the target constructible or emergent? Can the generator satisfy it by writing a specific token into the artifact, or must it find a computation whose behaviour lands somewhere? Inside one decoder, holding everything else fixed, this is worth **+47.7 points** [+34.8, +60.2] (§3.3).

Is the target observable to the generator? Can it check its own compliance, or is the success criterion computed somewhere it cannot see? A collaborating measurement on a glob-pattern decoder isolated this axis directly: asked to match a count over a 600-name corpus it could not see, compliance was 45.8%; shown the corpus, **90.0%**, with nothing else changed. Unlike tolerance, this axis moves accuracy rather than the return rate — the unobservable arm answered every time and was never right (§3.8).

Is the target a point or a band? Stating an emergent target as a range rather than an exact value is worth **+51.5 points** in the rarest band and nothing in the head (§3.4).

The three compose, and the worst corner is the one that matters commercially. A practitioner steering a closed-weight video model is usually asking for something emergent (a quality that comes out of the generation rather than being nameable in it), unobservable (they cannot see the model's output distribution, and the model cannot see their acceptance criterion), and stated exactly (*this shot must be 2.4 seconds*). Our own estimate for the middle case is 31.3%, and the honest extrapolation to the worst corner is *below* that, not at it.

Which yields a short procedure, in the order the interventions are cheap:

1. **Restate the target as a band** wherever the task tolerates one. One line of prompt, no extra generation, +51.5 points where it is needed and nothing where it is not. But put the tolerance in the ask — relaxing acceptance afterwards recovers nothing, because failures are wholesale rather than near-misses.
2. **Make the criterion observable.** Put the thing you are scoring against into the prompt if it will fit. This is what moved the glob decoder from 45.8% to 90.0%, and it costs context rather than calls.
3. **Decompose emergent targets into constructible ones** where you can. *A red door on the left of frame* is constructible; *the pacing of this edit rhythm* is emergent. If a target can be restated as something the generator writes rather than something it must search for, restate it.
4. **Budget the remainder.** What is left is emergent, unobservable and exact, and for those the compliance rate is low enough that the question becomes how many attempts you can afford — which is §5.

Both cross-terms in §3.4 were measured rather than inferred, and the design is what makes the prescription safe: without *asked exactly, scored on the band* a reader would reasonably assume they could get the benefit by loosening their evaluation instead of their prompt, and they would get nothing.

A caution we earned. We registered a prediction that a fifth decoder, SVG, would be the clean positive control for the first axis: every target is a mark the generator can simply draw. Compliance was **18.6%** against a registered floor of 70%, and by our own falsifier that test does not support the distinction (§3.6). The most likely reason is the second axis, and it is a mistake worth naming because it is easy to repeat: an SVG target in our scoring is a *conjunction of four quantized properties*, three of which are our measurement apparatus rather than anything the generator can compute. It

cannot tell whether the circle it drew crosses the area-fraction threshold separating our size bands. We built the unobservability into the scoring and then called it a positive control. **If you quantize your acceptance criterion, you have made your target unobservable, however constructible it looked.** The miss distances confirm it: 73.3% of the misses in which the required element was drawn at all are within one band of the target on all three quantized axes (§3.6).

3.6 A registered prediction that failed, and what the failure measured

We registered, before generating any data, that a fifth decoder would be a clean positive control for §3.3. SVG marks looked maximally constructible: a required `<circle>` in a named grid cell with a named size and colour is satisfied by drawing that circle. The registration set a floor of 70% compliance, predicted approximate flatness in rarity, and named its own falsifier.

registered	observed	
`instruct` ≥ 70%	**18.6%** [12.2, 25.0]	**failed**
approximately flat in rarity	13.9 / 11.1 / 30.6 / 16.7 / 25.0	**failed**
`decoy` near the floor, below `instruct`	3.2% vs 18.6%	held

By the registered falsifier, **this test does not support the constructible/emergent distinction**, and we report it that way rather than adjusting the prediction. The distinction rests on §3.3's within-decoder result, not on this.

What went wrong is the second axis, and it was our doing. Every other decoder's target is one property. An SVG target in our scoring is a *conjunction of four quantized properties* — tag, grid cell (1 of 36), size band (1 of 5), hue band (1 of 9) — and three of those quantizations are our measurement apparatus. The generator is told "column 5 of 6, medium in area, blue". It cannot compute whether the circle it drew has an area fraction above the 0.05 threshold separating our size bands, or whether its blue falls in our hue bin 5 or 6. We built the unobservability into the scoring and then called it a positive control.

The miss distances say so, and they discriminate where a rescore could not. A tolerance rescore lifts compliance under the rival explanation too, if enough mass happens to sit nearby. So we recorded the marks each generation actually produced and measured how far the misses fell, on each axis separately, starting with the one that cannot be explained by quantization at all: was the required element drawn?

misses in which the required tag was drawn at all	**76.5%** (101 of 132)
grid-cell distance, over those	median **0**; 79% in the exact cell, 11% adjacent
size-band distance	median 1; 47% exact, 39% off by one
hue-band distance	median 0; 50% exact, 36% off by one
within one band on all three axes	**73.3%** of tag-present misses

The generator draws the right element, in the right cell, in roughly the right colour and size — and our scoring rejects it for being one size band out. So SVG turns out to test **observability rather than constructibility**. That changes what the domain is evidence for; it does not retroactively make it evidence for the prediction it failed.

3.7 Miss distance tells you which axis you are failing on

Putting the two failure profiles side by side gives something neither produced alone, and it is directly usable.

decoder	how failures fail	
programs (emergent targets)	**wholesale**	asking exactly and scoring on a ±10% band gives 67.4%, indistinguishable from scoring exactly (66.7%) — almost nothing sits just outside the line
SVG (unobservably quantized targets)	**by a hair**	73.3% of tag-present misses are within one band on all three axes

The shape of the miss distribution is therefore a *diagnostic for which axis is binding*, and it costs one pass over generations you have already paid for:

- **Misses cluster just outside the criterion.** Your acceptance test is at a resolution the generator cannot verify. Widen the criterion or state the tolerance — the artifact is already almost right.
- **Misses are spread out, or the required element is absent.** The target is emergent, and tolerance will not save it. Decompose it into something constructible, or budget for the attempt rate.

These prescriptions point in opposite directions, and a single compliance number cannot tell you which one applies. Both of ours read 18.6% and 66.7% — neither number says whether to widen the criterion or rewrite the target.

3.8 Two mechanisms, separated

Tolerance and observability are easy to conflate — both are ways a target can be ”too demanding” — and two measurements separate them cleanly, each holding the other axis fixed.

Tolerance changes the return rate. On programs, the exact ask and the band ask hit at the same rate once the generator answers at all (97.7% against 97.3%); what differs is how often it answers (68.2% against 87.6%). Loosening a target does not help the model find a better artifact. It helps it produce one.

Observability changes accuracy. A collaborating measurement on a glob-pattern decoder asked for an exact match count over a 600-name corpus the generator could not see. That arm returned a pattern **24 times out of 24** and hit its target **0 times out of 24**. Full completion, zero compliance: the model was perfectly willing to answer and could not, because the information needed to answer was not available to it. Shown the corpus, compliance went to 90.0%.

	return rate	compliance	what is binding
exact ask, observable (programs)	68.2%	97.7% *of returns*	the model often declines to answer
exact ask, **un**observable (glob)	**100%**	**0.0%**	the model answers and cannot be right

These are different failures and they call for different repairs. A low return rate is fixed by loosening the ask. A zero compliance rate at full return is not fixed by loosening anything — the generator needs the information, and §3.5's second step is the one that applies.

The diagnostic is cheap and it is the return rate. **If your generator is answering and missing, the target is unobservable or emergent; if it is declining to answer, the target is over-tight.** A single compliance number cannot distinguish these, and both look like ”the model can't do it”.

One qualification on the glob figures: the corpus-shown arm returned 20 of 24 against the other arms' 24 of 24, a 17-point spread that exceeds the threshold this paper recommends. It runs against the arm with the highest compliance, so 90.0% is conservative rather than inflated, but it is reported rather than glossed.

4. Distances: the channel as a ruler

Everything in this paper so far is a statement about diversity. The measurement underneath it is not. A pipeline that embeds text and acts on the distances is making a bet that has nothing to do with diversity

in particular: that proximity in the embedding stands in for proximity in the artifact. Deduplication makes it, contamination screening makes it, nearest-neighbour retrieval makes it, coreset selection and active learning make it.

The natural way to check the bet is to decode a sample, compute a distance in the artifact's space, and correlate. This section shows that this check does not work — not that it is noisy, but that it returns the same answer when the artifact is replaced by something with no connection to it — and then measures the three decisions directly instead, which is what survives.

4.1 Decoders that leave nothing to interpretation

An image is rendered by a model, which is why every artifact-space number in the hosted-render setting is a statement about an unlogged seed distribution: the endpoint exposes no seed, so a render cannot be repeated and render noise cannot be separated from the quantity being measured. That bound is structural.

It is removed by choosing decoders that are *deterministic and total*. A Python program run on a fixed battery of inputs, a SQL query run against a fixed database, a regular expression matched against a fixed corpus of strings: each maps text to an artifact with no sampling anywhere, so artifact distance is a property of the text alone and repeats exactly. These are also among the largest synthetic-data industries there are.

domain	text	artifact	artifact distance	coverage
code	a Python function `f(xs)`	its results on a fixed 140-input battery	fraction of the battery on which two programs disagree	distinct (input, result) cells
SQL	a `SELECT` against a fixed schema	the rows it returns	Jaccard distance between result-row sets	distinct rows retrieved
regex	a pattern	the subset of a 210-string probe corpus it matches	Jaccard distance between matched sets	distinct strings matched
library inputs	a string handed to `ipaddress`	the arcs of the library it executes	Jaccard distance between arc sets	distinct arcs reached

None of these distances is computed with reference to an embedding, so each can adjudicate an embedding rather than agree with it. The first is the corpus used below: 553 generated Python programs, of which 203 are distinct sources, which between them exhibit **39 distinct behaviours**.

4.2 The profile that looks like a finding

Correlating text distance against behavioural distance *within* each decile of text distance, rather than pooling, produces a table that appears to say something sharp:

decile of text distance	width	pairs	corr. with behavioural distance	mean behavioural distance
1 (closest)	0.140	2,051	**+0.451**	0.549
2	0.032	2,050	+0.112	0.827
3	0.022	2,050	+0.010	0.877
4–9	0.014–0.018	12,301	−0.040 … +0.062	0.878–0.909
10 (farthest)	0.104	2,051	**+0.065**	0.916
pooled		20,503	+0.364	0.853

Read on its own this says: the embedding tracks behaviour among near-duplicates and stops carrying information beyond them. The conditional mean of behavioural distance given text distance says the same thing without any correlation at all, rising **+0.474 across the near half of the distance range and +0.036 across the far half**. Every diversity selector in the literature draws its picks from the top decile.

We believed this, and it is wrong.

4.3 The control: two encoders, no decoder

The profile compares one distance matrix to another and reports that they agree more among close pairs. Nothing in that description mentions a decoder. If any two distance matrices over the same points agree preferentially in the near field, the shape is a property of the comparison and the decoder is doing no work at all.

Running the identical profile between two *encoders*, with no decoder anywhere in the computation:

comparison	near-field corr.	far-field corr.	**near − far**
nomic vs behaviour *(the claim)*	+0.451	+0.065	**+0.386**
char TF-IDF vs behaviour	+0.474	−0.002	+0.476
word TF-IDF vs behaviour	+0.513	−0.053	+0.567
nomic vs char TF-IDF *(control)*	+0.742	−0.003	**+0.745**
nomic vs word TF-IDF *(control)*	+0.750	+0.007	**+0.742**
char TF-IDF vs word TF-IDF *(control)*	+0.898	+0.117	**+0.781**
nomic vs *permuted* behaviour *(null)*	−0.002	+0.023	−0.025

The decoder-free controls show the shape about twice as strongly as the decoder does, and this holds on every decoder we have measured. A second decoder — 603 generated SQL queries scored by the cells they return — at first appeared to be the exception, with a decoder profile of +0.231 against cross-family baselines of −0.066 and +0.037. It is not. Bootstrapping over items (300 resamples), its margin over its *largest* baseline is **−0.195, 95% CI [−0.337, +0.013]**, positive in 5.0% of resamples. On three decoders now — Python programs, SQL queries, and strings handed to a library — the decoder profile fails to clear the encoder-to-encoder baseline. The confound explains the whole of every correlation we have. The conditional-mean curve is the same story: the control rises +0.493 over the near half and +0.030 over the far half, against the decoder's +0.474 and +0.036 — the two curves are the same curve. Two representations that have never seen a program run produce a cleaner ”near-field law” than the comparison against what the programs actually do.

A third of the effect is bookkeeping. Equal-count deciles have very unequal widths — the first spans 0.140 of the distance range and the eighth spans 0.014 — and a within-bin correlation is attenuated by the bin's own spread. On equal-width bins the profile is +0.288, +0.056, +0.114, +0.191, +0.061, −0.004, +0.014, +0.030, +0.070, −0.080: still more signal at the low end than the high end, and nothing like the monotone decline the decile table displays.

One control does pass, and it is the one that matters for what remains. Permuting the artifacts across items — breaking the text-to-behaviour correspondence while leaving both marginal distance distributions exactly intact — flattens everything to −0.025 near-minus-far and −0.003 pooled, against +0.364 unpermuted. **There is a real association between what a program looks like and what it does.** What there is no evidence for is the shape we read into it.

4.4 The consequence: correlational proxy validation is confounded

This generalizes past our own mistake, because the check we ran is the check the field runs. A paper that proposes an embedding-based proxy for something expensive — human preference, rendered quality, downstream accuracy, behavioural diversity — validates it by decoding a sample and reporting a correlation, and frequently reports that the correlation is strongest among similar items. That last observation is not evidence. It appears between two arbitrary encoders that have never seen the outcome.

The prescription is cheap and almost nobody follows it:

> **Report a decoder-free control beside any proxy-validation correlation.** Correlate your embedding against a second, unrelated representation of the same corpus. Whatever agreement survives *above* that control is the part your artifact measurement contributed. On the corpus here, that residue is negative: the decoder profile is weaker than the encoder-to-encoder baseline.

And the positive form: **validate a proxy by running the decision, not by correlating the distances.** A correlation is a summary of a geometry; a decision is a thing that either works or does not, is scored

in the artifact's space, and has no confound of this kind. The rest of this section does that for the three decisions a synthetic-data pipeline actually makes.

4.5 Decision 1 — deduplication is a corpus instrument, not an item gate

Deduplication reads the closest pairs, which is where whatever signal exists is concentrated, and at corpus scale it works: duplicate-pair detection on the code corpus runs at **AUC 0.779** against the ground-truth label *identical on the whole battery*.

Per item it does not. Among the pairs a near-duplicate filter would reject, the fraction that are behaviourally *far apart* (distance > 0.5):

filter radius	pairs rejected	of those, still behaviourally far	mean behavioural distance
5th pctile	1,026	**45.4%**	0.419
10th	2,051	62.2%	0.549
20th	4,101	76.8%	0.688
30th	6,151	83.0%	0.751
40th	8,201	86.3%	0.784
pool-wide			*0.853*

At the tightest radius, nearly half of what the filter discards behaves unlike the item it was discarded for resembling; the best radius available reaches F1 0.449 (precision 0.452, recall 0.446). A second decoder — 202 strings scored by the `ipaddress` arcs they execute — gives 41.6% at the same quantile against a pool-wide mean of 0.672.

This is a direct measurement of the decision, so the §4.3 control does not touch it. Threshold tuning does not fix it either, because the error is in the strength of the evidence rather than the location of the boundary. **A text-space duplicate filter is defensible as a corpus-level instrument — a duplicate rate, a contamination screen — and not as a per-item accept/reject.**

4.6 Decision 2 — diversity selection has no stable sign, and both signs lose to the oracle

Farthest-point traversal draws its picks from the top decile. What that costs turns out to depend on the decoder, and not in a way anything visible in the text predicts. On the `ipaddress` decoder, greedy max-min is worse than random at every budget in every representation, by 54 to 82 arcs. On the code corpus it is worse than random at $k = 10$ and better from $k = 20$ on:

k	random	max-min	best near-duplicate filter	**artifact-space oracle**
10	815	743	849	**1,336**
20	1,119	1,296	1,267	**2,047**
30	1,355	1,509	1,517	**2,365**
50	1,549	1,864	1,809	**2,414**
100	1,891	2,358	2,113	**2,414**

Behaviour cells covered, mean of 20 seeds; the oracle is greedy marginal coverage computed in the artifact's own space.

Two things follow. **Whether text-space diversity selection helps is a property of the decoder and is not predictable from the embedding** — two decoders, the same selector, opposite signs. And **the gap that matters is against the oracle, not against random**: the best text-space selector at $k = 10$ buys 849 cells where the oracle buys 1,336, the oracle reaches at $k = 30$ what text-space selection needs $k = 100$ to reach, and it saturates the pool at $k = 50$ where no text-space arm saturates at all. Text-space selection leaves between a third and a half of the reachable behaviour unbought at small budgets whether or not it beats random.

The oracle is normally waved away as the thing one cannot afford. §8 measures what it costs.

4.7 Decision 3 — the redundancy that no text-space instrument reaches

The corpus itself makes the case for measuring the artifact more compactly than any correlation does. Naive repeated prompting produced 553 programs. Removing byte-identical sources — the deduplication step every pipeline performs — leaves **203**. Those 203 distinct programs exhibit **39 distinct behaviours**.

So text-level deduplication, performed perfectly, removes 63% of the corpus and leaves it **5.2× redundant in the space that matters**. The residue is not subtle paraphrase. One pair of programs in it implements the longest-increasing- subsequence length by patience sorting with a binary search, and the other by the quadratic dynamic program; they share almost no tokens, and they agree on all 140 battery inputs. No embedding-space threshold reaches that pair without also discarding most of the corpus, and running both programs takes microseconds.

5. Measure with the decoder: it is cheaper than the channel

The argument for computing distances on text rather than on the artifact is always the same, and it is never stated as an argument because it is assumed: decoding is expensive, embedding is cheap, so one embeds. For a rendered image or a two-minute instrumental that is true. For the decoders in this section it is false, and by a wide margin.

Measured on one laptop, per item:

operation	ms per item	relative
decode: string → `ipaddress` arcs, plain execution	**0.11**	1×
decode: same, with `sys.settrace` instrumentation	**0.98**	9×
decode: Python program → 140-input battery, **each in a fresh OS process**	**29.1**	265×
embed: `nomic-embed-text`, local Ollama, batched	107.9	981×
embed: `nomic-embed-text`, local Ollama, one at a time	1,488.2	13,529×
embed: character TF-IDF, local, batched	18.3	166×

The code row is deliberately the least favourable decode measurement we can honestly make: every generated program is run in a *separate operating-system process* for isolation, so Python interpreter start-up dominates and the actual execution is a rounding error. Even so, decoding is **3.7× cheaper than a batched local embedding** and **51× cheaper than embedding items one at a time**, which is what a streaming pipeline does. Where the decoder does not need process isolation the margin is three orders of magnitude.

Two honest qualifications. Instrumentation, not the program, is the dominant decode cost — tracing costs 9× plain execution — so a decoder whose instrumentation is heavy needs its own measurement rather than an inherited ratio. And the embedding figures are for a *local* model with no network hop and no per-token charge, which is the best case for the text-space route; a hosted embedding endpoint is slower and has a price.

The conclusion is not rhetorical. **For a deterministic decoder that runs in milliseconds, the artifact-space oracle is not the expensive option a practitioner forgoes. It is the cheap option they did not think to run**, and it recovers between 1.02× and 1.64× the coverage of the best text-space selector at matched budget (§4.6), reaching at $k = 30$ what text-space selection needs $k = 100$ to reach.

Where decoding genuinely is expensive — rendered images, audio, anything behind a paid endpoint — the fallback is not a text-space selector but a *partial decode budget*: decode a sample, and spend the rest of the budget where the sample says the behaviour is. That regime is where §5's cross-modal steering belongs, and it is the boundary between the two halves of this paper.

5.1 The decode budget: what a partial decode buys

For a cheap decoder the advice is simply to decode everything, and §5.1 shows it costs less than embedding. That advice is useless where the decoder is a video model, an image model behind a meter, or anything else charged per call. There the question is not whether to decode but **how much**: *I can afford to decode m of my N candidates — what do I get?*

Decode a uniformly random m of the pool, run the artifact-space greedy selection restricted to those, and score the chosen k in the artifact's space. We report the fraction of the oracle's advantage over random that this recovers,

$$\text{recovered}(m) = \frac{\text{partial}(m) - \text{random}}{\text{oracle} - \text{random}}$$

which is 0 when decoding buys nothing and 1 when a partial decode is as good as decoding everything. The text-space selector is scored on the same scale, since it is what a pipeline does when it decodes nothing at all.

decoder	k	text-space max-min	m = 5%	10%	20%	40%	70%
programs (N = 203)	10	−10%	0%	47%	69%	83%	93%
	20	18%	0%	0%	35%	66%	85%
	30	17%	0%	0%	10%	45%	76%
	50	36%	0%	0%	0%	29%	67%
SQL (N = 603)	10	16%	40%	56%	73%	90%	101%
	20	24%	16%	40%	61%	82%	95%
	30	26%	0%	29%	53%	77%	93%
	50	27%	0%	8%	39%	68%	90%

Percentage of the oracle's advantage over random recovered, mean of 40 seeds.

Three things a practitioner can use.

A little decoding beats a lot of embedding. Decoding a fifth of the pool matches or beats text-space selection at every budget on both decoders, and decoding two fifths roughly doubles it. The text-space column is what the alternative buys, and it never exceeds 36%.

The budget that matters is relative to the pool, not to the selection. Recovering 70% of the ceiling took m between 4.7× and 12.1× the selection budget across our rows, which is too wide a range to publish as a rule. Expressed as a share of the pool it is stable: **decoding 40% of the pool recovers 29–90% and decoding 70% recovers 67–101%**, with the low end of each range belonging to the largest selection budget. The reason is structural — a greedy selector choosing k items from a decoded sample of m has only m/k candidates per pick, so the sample must be several times the selection budget before the greedy choice has anything to choose between.

Select small, or decode more. The recovered fraction falls monotonically in k at every m. If the decode budget is fixed and small, taking fewer items from it is better than taking more, which is the opposite of what a fixed-size corpus requirement encourages.

One boundary is worth stating plainly. A collaborating measurement on a decoder with microsecond decoding and a knowable ceiling recovered 72–81% of the advantage at m = 40%, better than our programs decoder at the same fraction. The spread across decoders is real and we do not have enough of them to model it, so the table above should be read as the shape of the curve rather than as constants to plan against. Running it on your own decoder costs one afternoon and is what `decodergap` exists to do.

6. `decodergap`: the tool

The results above do not compose into a rule that can be applied blind. Whether text-space selection helps varies in sign by decoder; whether a target can be steered depends on three properties of the target; and how much decoding to buy depends on a curve that differs across decoders by enough that we publish it as a shape rather than as constants. What they compose into is a set of measurements cheap enough to run before a pipeline commits, and `decodergap` is those three measurements behind three calls.

```
import decodergap as dg
```

```python
# 1. Is my text-space DISTANCE machinery sound for this decoder?
rep = dg.audit(texts, embed, decode, distance, coverage)
print(rep.summary())

# 2. How should I state a target I want the artifact to satisfy?
print(dg.triage("the shot must be exactly 2.4 seconds long"))

# 3. I can afford to decode m of N candidates. What do I get?
print(dg.plan(decode_budget=2000, pool=5000, select=50))
```

`audit` runs §4 on your corpus: the near-field profile, the decoder-free control, both deduplication verdicts and the selector comparison against the artifact-space oracle. Three design decisions in it are findings rather than preferences.

It never reports a correlation without its decoder-free control. The profile of §4.2 looks like a finding and is reproduced more strongly by two encoders that never saw a decoder (§4.3). Every proxy-qualification number the tool emits is accompanied by the encoder-to-encoder baselines it has to clear, and it must clear the whole distribution rather than a member of it — selecting which baseline to compare against is how we lost a result of our own (§10).

It issues two deduplication verdicts. The aggregate verdict asks whether a radius can rank duplicate pairs at all; the per-item verdict asks what fraction of the pairs it would reject are behaviourally far apart. On both decoders measured the first passes and the second fails, at 45.4% and 41.6%.

It scores every selector in the artifact's space and against the oracle, so a text-space selector cannot win by agreeing with the representation it selected in, and beating random is not mistaken for doing well.

It declines to recommend a filter radius. We swept the threshold over the 5th to 40th percentiles at four budgets: the best cell is +20.7 arcs over random at one budget, and by $k = 50$ every threshold loses. Any single-budget experiment would have produced a shippable-looking constant.

`triage` classifies a target on the three axes of §3.5 from its wording and says what to do about it. It is a heuristic over the phrasing rather than a measurement, so it is a prompt for the practitioner's judgement — but the axes it checks were each isolated by a separate experiment, and the intervention it suggests for an emergent exact target is the one worth +51.5 points in the tail. It also warns, on every target, to check observability separately, because that is the axis we lost a registered prediction to.

`completion_check` refuses a cross-arm verdict when the arms did not answer at equal rates. This is in the tool because it caught a 19-point effect of ours that was not there (§3.4) and a boundary result of a collaborator's that rested on five observations out of sixty. In both cases the tell was the sample count rather than the effect size, and in both cases the spurious result was cleaner and more interesting than the truth.

`diagnose_misses` reads §3.7 off generations you have already paid for. Given the failures and a function returning how far each fell on each axis of your acceptance criterion, it returns the profile and a verdict: *unobservable criterion* when the misses cluster within one step on every axis, *emergent* when they are spread or the required thing is absent. On our SVG failures it returns the first, at 73.3% within one step and the required element present in 76.5%.

`plan` turns a decode budget into an expected recovery, interpolating the curves of §5.3, and warns when the budget is fewer than about three decoded candidates per pick — the regime where greedy selection has nothing to choose between and recovers close to nothing. Its constants are two decoders' worth of measurement and are meant to be replaced: running `audit` on your own decoder substitutes yours.

7. When the decoder is expensive

Everything above rests on decoding being cheap. Where the decoder is a diffusion model or a two-minute audio generation, the oracle of §5 is genuinely out of reach and the control modality is all a

pipeline has. This section reports what the same questions look like there, on rendered-image and rendered-audio corpora. The answers are worse, and they are worse in the direction §3 predicts.

7.1 Text diversity is nearly blind to image diversity

We rendered the first 199 DALL·E instructions from the naive corpus and embedded them with CLIP. Pairwise cosine similarity in text-embedding space correlates with pairwise cosine similarity in image-embedding space at **Pearson *r* between 0.167 and 0.421** across the seven rendered arms, and at 0.285 within-arm centred. The gap varies by a factor of 2.5 depending on which corpus is measured: lowest on `self_instruct` (0.167) and on the naive arm (0.170), highest on the arm this paper's own method produced (0.421). Pooled, the shared variance is 14.5%; within-arm it is 8.2%. Knowing that two instructions are semantically far apart therefore tells you rather little about whether the two pictures look different, and every text-side diversity method in this literature — ours included — is optimizing a proxy that leaves most of what the reader receives unexplained. That the correlation is strongest on our own corpus is worth stating plainly: the proxy is least misleading exactly where the instructions were built to differ structurally rather than only lexically.

The qualitative version is more damning than the correlation. Independently generated instructions, from a corpus with a 0.000 exact-duplicate rate and healthy lexical diversity, render to near-interchangeable pictures: the same magenta-and-cyan palette, a classical marble bust, neon signage, halftone collage, a receding grid. A vision judge shown samples rates the set's distinctness 6–7 out of 10 and names the attractors precisely — *"muted beige, cream, brown, ochre and black foundations accented by saturated cyan/teal, turquoise, pink"*, *"appropriation of canonical or religious imagery, especially Mona Lisa-like female portraits"*, *"frontal, museum-like presentation with centered, symmetrical compositions"*. This is a second mode collapse, downstream of ours, contributed by the image model and by the fact that much of what varies in the text lands in the same visual place.

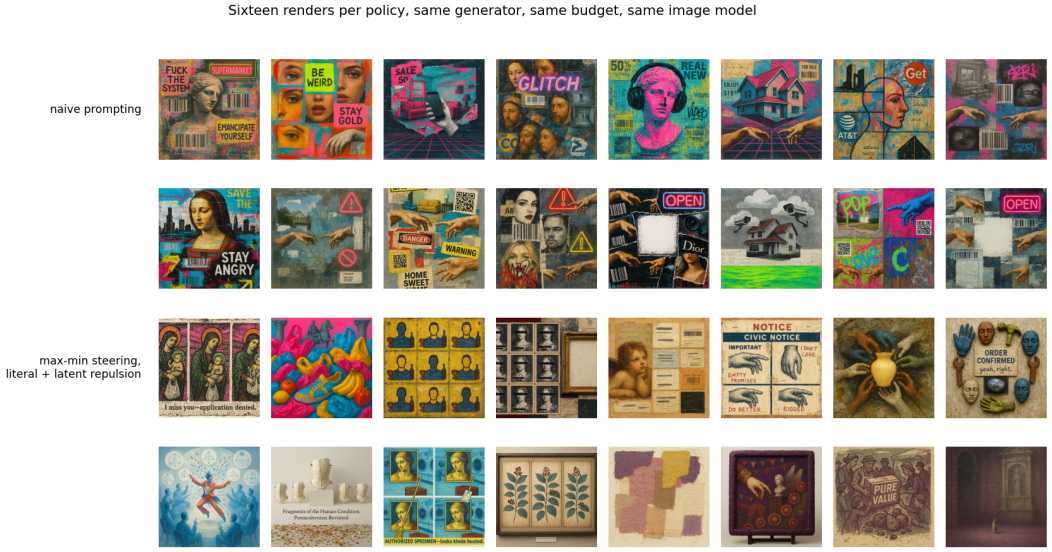

Figure 1. Sixteen renders per policy, same generator, same budget, same image model. Rows 1–2, naive prompting: a corpus with a 0.000 exact-duplicate rate and healthy lexical diversity that still returns one visual mode — magenta and cyan collage, barcodes and QR codes, Renaissance portraits and classical busts, Michelangelo hands, warning triangles and neon OPEN signs recurring across nearly every panel. Rows 3–4, max-min steering with literal and latent repulsion on both the text and vision sides: medium, palette, register and composition all move, across a medieval triptych, a botanical cabinet, a photographed sculpture installation, a torn-paper abstract and a civic notice.

7.2 Audio: whether a prompt can be steered is a property of the embedder

The more portable finding is that whether a prompt can be steered before rendering is a property of the *embedder* rather than of music. Measuring the Spearman correlation between prompt-pair similarity through the text tower and track-pair similarity through the audio tower:

embedder	alignment (naive / conditioned)	note
MuQ-MuLan	**0.68 / 0.47**	a usable steering gradient
CLAP-fused	0.50 / 0.21	weak; ~0.05 on the steered corpus
CLAP-music	0.18 / 0.06	worst; median within-corpus NN distance 0.004 — it hears one track

Cross-embedder agreement on pairwise track similarity spans 0.22–0.84, per-arm diversity verdicts flip between embedders, and each embedder nominates a different most-redundant pair. Two prescriptions follow: steer music with MuQ-MuLan rather than CLAP, and never publish an audio-diversity number without naming its embedder. Structural contracts have a length ceiling as well — compliance with a requested section sequence is exact at prompt lengths near 560 characters, 0.11 at a mean of 658, and zero by ~1,600 — so a structural contract for music must be short or it is noise.

7.3 The ranking does not depend on the viewpoint

A max-min score is a distance in a chosen embedding, so a natural worry is that it names a property of the embedder rather than of the corpus. Six exam-item corpora — this method's bank against naive sampling, persona prompting, self-instruct, evol-instruct and high-temperature sampling, 200 items each — were scored under four independent representations: CLIP text, TF-IDF with no learned semantics at all, the twenty-two-feature prosodic vector, and a 150-dimensional function-word profile of the kind used for authorship attribution. Each score is the fifth-percentile nearest-neighbour distance divided by the mean pairwise distance of the same cloud, which is invariant to the global rescaling an embedding choice is free to apply, and each representation is admitted only if its between-corpus spread exceeds its own within-corpus sampling noise — all four clear that bar by factors of 9.7 to 24.7.

The four views agree, at a mean Kendall tau of +0.83 across their six pairings, and **RAC ranks first under every one of them**, including the three it never optimizes. It also holds the best worst-case score, so it wins under a distributionally-robust reading of diversity as well as under any single view. The agreement is not an artifact of the degenerate baselines: restricted to the three corpora whose fifth-percentile distance is non-zero in every representation, mean tau is +0.67 and RAC is still first under all four. Naive sampling and high-temperature sampling score exactly zero on prosody and on stylometry — at least one item in twenty has an exact neighbour in those channels.

7.4 A class of repetition the embedding misses

One comparison in the artifact channel runs against the method, and it belongs here. Embedding metrics miss an entire class of visual repetition — tiled grids of one cell, a shared palette, the same composition recoloured — so we audit it with deliberately dumb non-semantic signatures: a 16×16 luminance layout map, an autocorrelation tiling score, a hue histogram. Scored at matched n = 60 and matched render tier, **the published baselines are better than our arms on both structural measures**: layout twins above 0.5 cosine average 0.223 across the five baselines against 0.473 across our eleven, and literal tilings 0.117 against 0.406, with no baseline worse than our best arm on layout. A corpus told to differ along seven or eleven named dimensions apparently converges on a compositional template that plain repeated prompting does not. Adding literal-space structural bans to the prompt — grids banned when recent renders tile, dominant hue pairs named and banned, layout-change demands when layouts collide, plus a learned text→bad-structure bridge — halves palette twins (0.78 → 0.35 in the coverage arm) and improves the coverage objective 21% (0.206 → 0.247), but does not close it. Optical spread and structural repetition are both measured on pixels and point in opposite directions: our corpora occupy a wider region of that space while repeating their compositional scaffolding more often within it.

7.5 A defect no diversity measure can see

The audits of §6 ask whether the axes reach the artifact. This one asks whether the resulting bank is *usable*. For a test bank, key position is a hard requirement: correct answers must be distributed across option positions, because an examinee who notices an imbalance can exploit it without reading anything. Asking a fixed solver each item and recording which position it chose:

bank	A	B	C	D	χ^2 vs uniform (3 df)
RAC	**0.900**	0.050	0.050	0.000	**90.4**
naive	0.225	0.550	0.225	0.000	24.6
RAC, after balancing	0.200	0.175	0.300	0.325	**2.6**

Ninety percent of the RAC bank's keys sit in position A, against a critical value of 7.81 at $p = .05$. An examinee who answers A to everything scores 90% without reading a stem, and neither bank ever places a key in D. This is a property of the solver only if the solver is not reading the items, so we separated the two by permutation: ask each item twice, once as written and once with its options reordered, and record which option *text* is chosen. The answer follows the content in 95% of RAC items and 100% of naive items, and stays on the same letter in only 17.5% and 30%. The solver reads; the imbalance is in the bank.

Why the method cannot see this. Key position is not a semantic property, so it is invisible to every measure in this paper — embedding diversity, *n*-gram diversity, Vendi, coverage and the enemy-item radius are all exactly as favourable on a bank whose key is always A as on a balanced one. It is invisible to the axis set for the same reason: all seven axes condition on item content, and even *distractor logic* governs what the distractors are like rather than where the key sits among them. And it is structural rather than accidental — a generator asked for a stem and four options writes the answer it has in mind first and builds distractors around it, so A is where the key lands by default.

The repair needs no answer key. Permuting each item's options uniformly at random moves the key with its own text from whatever position it held, so the bank becomes uniform by construction; options whose text refers to a position (”none of the above”, ”both A and B”) are detected and left as written, which affects 37 of 1,999 items. Applied to the RAC bank this takes χ^2 from 90.4 to 2.6 — indistinguishable from uniform — while content-tracking is unchanged at 95%.

The general form is the sharpest limitation in this paper. **A diversity objective defined over an embedding is blind to any property of the artifact the embedding does not encode, including properties that decide whether the artifact can be used at all.** Difficulty is the same story in the same domain: it is the parameter an item bank exists to span, it is latent rather than semantic, and nothing in the objective can see it. Measuring diversity well is not the same as measuring fitness for purpose, and the gap between them is not visible from inside the objective.

7.6 Steering through the proxy where it is all you have

7.6.1 Images: closing the loop where the product lives The vision-steered arm closes the loop where the product lives. It renders a bounded sample of accepted instructions, embeds them with CLIP, and feeds two things back into the text-side loop: a least-squares map from the crowded *image* directions into instruction-embedding space, so the text-side orthogonality term can push away from visual redundancy it cannot itself perceive; and mined *visual* attractors from the vision judge, appended to the same ledger as the textual ones. It is the paper's mechanism applied one level down — the ledger already repels against what the model keeps saying, and now also against what it keeps showing — and it is the best arm of the seven in that comparison.

The margin is measurable in the space the loop optimizes and visible on the page. At matched $n = 200$ in the DALL·E domain, the text-only RAC arm scores centered Vendi 77.67 and median nearest-neighbour distance 0.171; the vision-steered arm scores 91.97 and 0.192, an 18% gain in centered Vendi, and leads every column of the seven-arm comparison — distinct-2 0.699 against 0.660, 4-gram self-repetition 0.040 against 0.068, *n*-gram Vendi 170.9 against 167.0. The contact sheet of §7.1 shows the same thing without a number: rows 3–4, steered on both the text and the vision side, move medium, palette, register and composition where rows 1–2 return one visual mode.

7.6.2 Audio: cross-modal steering through an audio embedder At matched *n*, matched prompt length and the same generator, the conditioned music arm reaches centered Vendi 43.03 against naive's 22.99 (1.87×), halves 4-gram self-repetition (0.140 against 0.291), raises distinct-2 (0.545 against 0.348) and holds roughly three times the room between nearest neighbours (0.079 against

0.027), on 100 prompts per arm with no exact duplicates in either — so the effect is semantic. On 100 rendered Lyria instrumentals generated by cross-modal steering with zero rejection, 100% of tracks verify as instrumental in CLAP space and mean-centered CLAP Vendi is 17.43 against 11.5 for naive prompts and 14.1 for axis-conditioned prompts under the same embedder.

The rendered arm was steered through CLAP and is measured in CLAP. §7.2's alignment table says the same loop through MuQ-MuLan would have a steering gradient nearly four times as strong on the naive arm (0.68 against 0.18), which is the experiment we would run next; the prescription — steer music with MuQ-MuLan rather than CLAP — is §7.2's, and the loop is the image one of §7.6.1 with the audio tower in place of CLIP.

7.6.3 Best-of-*K* at the render: select on the channel the objective cannot see Measured on the written candidate rather than the render: selecting on the gap alone raises the minimum nearest-neighbour distance by 20% in three of three seeds, at a cost of a third of a craft point. The same question at the *render* has a sharper answer, because the two channels have very different effective dimensions. Best-of-*K* on the written candidate happens before the handoff and §6 locates the loss after it, so we also ran $K = 3$ renders behind each of 60 fixed instructions and kept the render maximizing min distance to the accepted set — once in CLIP, once in optical statistics, over the same candidates. Selecting on optics raises pixel min NN from 1.039 to 1.292 (1.24×) and the fifth-percentile NN by 0.336 (paired subsample, 95% CI [+0.128, +0.488]), **and costs nothing in the channel the method optimizes**: CLIP min NN moves by −0.001 with the interval [−0.029, +0.033] tight around zero. Selecting on CLIP raises CLIP min NN by 0.040 ([+0.025, +0.071]) and moves optics not at all detectably. Both gains lie on the $K^{1/m}$ curve — 1.24× at $K = 3$ implies $m \approx 4.4$, and 1.04× implies $m \approx 17$ — so render-side selection is oversampling rather than a new mechanism. What the dimensions add is an allocation rule: oversampling buys far more in a low-dimensional channel than in a high-dimensional one, and **if a pipeline is going to pay for extra renders, it should select them on the channel the objective cannot see.**

7.7 Conditioning across the seam

Whether a commanded attribute shows up in the artifact can be audited by manipulation rather than correlation: hold a base spec fixed, set one axis to each of its levels in turn, generate, and read the displacement, centring within base so that only variation produced by *setting* the axis contributes, with a null that shuffles levels within each base. Two instruments read each artifact — *identifiability*, whether CLIP(ℓ) is the nearest of the axis's level descriptions to an image generated under level ℓ, against a chance rate of 1/|levels|; and *separation*, the η^2 of artifact embeddings grouped by commanded level under a permutation null — and a third channel that lives outside the embedding the method optimizes: pixel statistics for images, prosody for poems. On poems every axis is realized on at least one channel. On images three of seven axes identify their level at or below chance, two are undetectable on both channels, and the axis the attribution scoring ranked *first* is the least realized in the corpus. The image domain is where the proxy question becomes a conditioning question, because the image pipeline has a seam the poem pipeline does not.

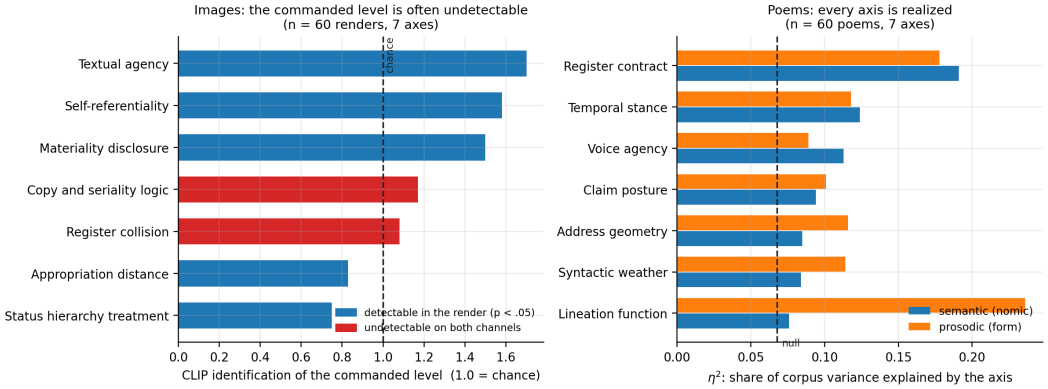

Figure 2. Axis realization measured on the artifact. Left: on 60 max-min renders, three of seven axes identify their commanded level at or below chance, and two are undetectable on both the CLIP and pixel channels. Right: on 60 poems, every axis is realized — on the semantic channel, the prosodic channel, or both. Dashed lines mark chance and the permutation null. **Images, by intervention: the loss is at the render, not in the writing.** The image pipeline has two stages — the model that proposed the axes writes an instruction, and `gpt-image-1-mini` renders it — so the same manipulation can be read on both sides of the handoff. Over all eleven axes at three bases and five levels, embedding the written instruction with CLIP, the render with CLIP, and the render's optics separately: nine of eleven axes are realized in the **written instruction** at $p < 0.05$ (mean $\rho = 0.148$), against four of eleven in the **render** (mean $\rho = 0.096$). Ten of the eleven attenuate, a mean loss of 35% of the text-side effect (Wilcoxon signed-rank $p = .005$; paired t, $p = .003$, both two-sided). **Conditioning reaches the instruction and loses a third of its grip at the render.** The image domain's weak realization is therefore not a failure of the axis calculus to produce usable conditioning — the conditioning is there, in the prompt, and measurable — but of that conditioning to survive a generator that never saw the axis set.

Part of the apparent loss is the instrument rather than the renderer, and this is the §7.1 lesson arriving from the other direction. The four perceptual axes attenuate most in CLIP and are precisely the axes CLIP is least equipped to register: measured on pixels instead, *Lighting direction* reads 0.229 against 0.092 in CLIP, *Shot scale* 0.252 against 0.105, and *Motion and temporal blur* 0.203 against 0.069. An axis commanding optics moves the optics and barely moves a semantic embedding.

The seam explains the split. The calculus is identical across the two domains, the axes are comparably abstract, and the budgets match. What differs is where the conditioning has to travel. A poem is written by the same model that proposed the axes, so the conditioning never leaves the system that understands it. An image axis is written by that model into a prompt and handed to a different model, which never saw the axis set, shares no latent space with it, and is under no obligation to honour a distinction like *copies outrank an absent original*. §7.1 measured this seam from the outside as a proxy gap; the intervention measures it from the inside. **Conditioning strength degrades at the boundary where the generator changes hands**, so the modality-agnostic claim holds for the calculus — which needs only an embedding — and not for the conditioning, which needs a generator that reads the same language the axes are written in.

8. Practice

The prescriptions are short, and each is attached to the measurement that motivates it. The first four are the ones we would keep if we could keep four.

1. **Never report a proxy-validation correlation without a decoder-free control.** Correlate your embedding against a second, unrelated representation of the same corpus, and against a permuted version of your artifact measurement. Whatever agreement survives above the first is what the artifact contributed; the second tells you whether any association exists. On our corpus the encoder-to-encoder baseline was **twice** the decoder signal (§4.3), which retires the finding we had.
2. **Qualify a proxy by running the decision, not by correlating the distances.** A correlation summarizes a geometry and inherits that geometry's confounds. A decision is scored in the artifact's space and either works or does not: what fraction of the pairs your filter drops are genuinely duplicates, how much artifact coverage your selector buys against the oracle.
3. **Split every deduplication claim into a corpus claim and an item claim.** They have different evidence and on both decoders here they get different verdicts: AUC 0.779 for ranking duplicate pairs, and 45.4% of rejected pairs behaviourally far apart at the tightest radius (§4.5). Contamination screening is defensible; a per-item gate is not.
4. **If your decoder is deterministic and fast, do not select in text space at all.** Decoding cost 29.1 ms per item against 107.9 ms to embed one, in the least favourable measurement we could construct (§5), and the artifact-space oracle reached at $k = 30$ what text-space selection needed $k = 100$ to reach (§4.6). The oracle is the cheap option, not the expensive one.
5. **Name the embedder, and publish the pairwise-similarity distribution it operates on.** Per-

arm audio verdicts flip between embedders and each nominates a different most-redundant pair (§7.2); mean pairwise cosine was 0.883 in one text corpus and 0.444 in another under the same embedder. A diversity number without its embedder and its similarity distribution is not a number.

6. **Count exact duplicates before computing anything, and report the literal and the latent separately.** They move in opposite directions in the same corpus (§7.3), and a 74% duplicate rate makes every distance statistic a statistic about duplication.
7. **Measure at the level of the artifact you ship.** Text-embedding similarity explains 14.5% of the variance in whether rendered images look alike, pooled across seven arms, and 8.2% within-arm (§7.1). If the artifact is rendered, the objective belongs in the space the artifact occupies.
8. **Audit in at least one channel the objective never optimizes**, with a distance statistic rather than a spectral one (§7.1). It is the only measurement that cannot be a restatement of the selection rule, and it is where the strongest positive result and the largest undetected defect in this line of work were both found.
9. **When a render is in the loop, steer through it.** Render a bounded sample, embed it in the artifact's space, and bridge its crowded directions back into the text space the loop optimizes (§7.6.1); steer audio through an embedder whose text and audio towers agree (§7.6.2).
10. **Spend extra renders on the low-dimensional channel.** Best-of-K buys $K^{1/m}$; selecting renders on nine pixel statistics buys 1.24× at $K = 3$ where selecting on CLIP buys 1.04× (§7.6.3).
11. **Measure axis realization at the artifact, by intervention, wherever the generator changes hands** (§7.7). Where the seam exists, a third of the conditioning does not cross it.
12. **Balance by construction what the embedding cannot see.** Key position, difficulty, compositional template: none is encoded, none is optimized, and each decides usability. A uniform permutation of options needs no answer key and takes χ^2 from 90.4 to 2.6 (§7.5); a structural ban block halves palette twins (§7.4).
13. **Read the output.** A corpus can be diverse along every dimension you thought to measure and uniform along the one you did not.

9. Limitations

- **One text embedder.** All text geometry is `nomic-embed-text` with cosine distance. §7.3 shows the *ranking* of methods survives four representations; whether the *magnitudes* transfer is untested, and the coverage and packing numbers inherit that.
- **A joint embedder varies both towers at once.** §7.2 changes one knob — the model supplying both the text tower and the audio tower — so it shows that the alignment moves and cannot say which side moves it. Crossing text-side and artifact-side encoders independently on fixed bytes is the decomposition this paper does not contain.
- **The renders carry no logged seed.** The image endpoint exposes no seed parameter, so no render in this paper can be regenerated, and render stochasticity cannot be separated from the quantity being measured. Every artifact-space number here — the proxy correlations of §7.1, the realization of §7.7, the render selection of §7.6.3 — is a statement about an unlogged seed distribution, and a repeat-draw floor at a logged seed is the missing measurement.
- **Several image results are single-seed.** The image arms are one seed each at $n = 60$; the vision-steered arm's margin in §7.6.1 is one run at $n = 200$; the audio steering arm is one run of 100 tracks.
- **Judge bias is unmeasured.** Vision and language-model judges name attractors, classify blueprint areas and read device forms; none is calibrated against human raters, and a judge that shares the generator's blind spots would miss exactly what the generator misses.
- **The structural signatures are deliberately dumb.** A 16×16 luminance map, a tiling score and a hue histogram find tilings and palette twins and nothing subtler; they are a floor on what the embedding misses, not a measure of it.
- **Steering closes part of the gap and not all of it.** Literal-space bans halve palette twins and improve the coverage objective by 21% and do not close the structural deficit; the vision-steered arm is the best arm in the image domain and still renders through a model that never

saw the axes.

10. What this cost us to learn

The confound of §4 is not an exotic failure mode. It is the default, and we fell into it repeatedly while writing the paper that describes it. Sorting our own results by whether they survived scrutiny makes the pattern hard to miss.

Survived — every one a direct measurement of a decision, scored in the artifact's space:

- the lever result and its decoy control (§3)
- the partial-decode budget curve, and the oracle intervals (§5)
- the cost comparison (§5)
- reject purity of a near-duplicate filter, 45.4% and 41.6% on two decoders (§4.5)
- max-min's standing against random, and the fact that its sign flips between decoders (§4.6)
- the decoder-free control itself, implemented independently in two sessions on different decoders within the same hour, agreeing

Died — every one an analysis of the shape of a correlation:

- the near-field law as a property of decoders, killed by the control in §4.3
- its apparent invariance across four encoders, which was the same artifact seen four times
- a within-decoder version of it, which survived until a permutation null put it at $p = 0.17$ — 17% of arbitrary splits of the same corpus produce a gap as large
- a refinement claiming dispersion rather than mean smoothness was the operative variable, which ordered the data backwards
- an early cross-representation result that a separability gate later showed could not have been measured at all, because three of five views could not rank the corpora above their own sampling noise

Six failures now, and the sixth is a different species. The tolerance main effect (§3.4) was real as an intent-to-treat number and not real as an accuracy claim: the exact ask returned nothing 32% of the time against the band ask's 12%, and conditional on the generator answering at all the two are indistinguishable at 97.7% and 97.3%. That is not a correlation-shape error. It is attrition correlated with the treatment, which is the classic way an experiment manufactures an effect, and we found it only because a collaborating session hit the same failure and suggested checking completion rates by arm.

Five failures of one kind, one survivor class, The failures were not bad luck: correlation shape is precisely the evidence type this paper argues is confounded, and we kept reaching for it because it is cheap, because it produces a table quickly, and because a decile profile with a monotone trend and $p = 3\times10^{-103}$ in its first cell looks like a finding.

One failure is worth stating in detail because the rule that catches it is general. The SQL decoder appeared to be the one case that cleared its baselines. It cleared *two* of them — the cross-family encoder pairs — and we discounted the third, the lexical-versus-lexical pair, as too easy a baseline to be meaningful. That was a justification constructed after seeing which baselines were inconvenient. Under the rule that a decoder must clear the whole baseline *distribution*, the margin is −0.195, 95% CI [−0.337, +0.013].

> **Choose your control before you see it, and clear the whole distribution of controls rather than a member you have selected.**

We would not have accepted the discounted baseline from anyone else, which is the usual sign.

11. Conclusion

You can address the generator in one modality and you need the output in another. The channel between them is real: permuting the artifacts destroys the association in every domain we measure, so text and artifact are genuinely coupled, and a corpus built by writing text is not writing into the void.

What the channel will not do is serve as a ruler. Distances measured in it do not stand in for dis-

tances in the artifact, the standard check that would catch this is confounded by a shape two arbitrary encoders produce between them, and the decisions built on those distances come apart under direct measurement — a duplicate filter that is sound for a corpus statistic and unsound per item, a diversity selector whose sign depends on a decoder nothing in the text reveals.

What it will do is serve as a lever. A specific artifact-space requirement, written into the text, is honoured 62.5% of the time against 6.9% for a decoy that states a different requirement in the same words — and the wrong requirement leaves the generator worse off than no requirement at all, which is how we know the channel is carrying the content rather than the form. How often it is honoured depends on where the target sits in the generator's own distribution: certain in the head, one time in three four bands into the tail.

The practical shape of that split is short. Steer through the channel, because that is the use it supports. Measure with the decoder, because for anything deterministic the decoder is cheaper than the embeddings it replaces and buys 1.54× the coverage at small budgets. And when you must qualify a proxy by correlation because the decoder is a diffusion model and you cannot afford it, run the decoder-free control first, on a baseline you chose before you saw it.

We did not run that control until the fifth attempt, and it retired four of our own results. That is the strongest argument we can make for it.